

Algorithm and Data Structure Comparison Tool

CSC506 Final Portfolio Presentation

Integrated system across search, linked structures, maps, hashes, priority queues, graphs, sets, Bubble Sort, and Quickselect.
Built to show theory, implementation, benchmarks, and recommendations as one final portfolio.

License: Apache-2.0

1

The following is a web application is an “Algorithm and Data Structure Comparison Tool” that compares search algorithms, linked structures, tree-backed maps, hash tables, priority queues, graph-based methods, sets, Bubble Sort, and Quickselect inside one Streamlit interface.

The project is part of the final portfolio that implements benchmark evidence, written analysis, and recommendation guidance.

Presenter Information

Alexander Ricciardi
Colorado State University Global
CSC506 - Design and Analysis of Algorithms
Professor Dr. Jonathan Vanover
May 10, 2026

2

I am Alexander Ricciardi. This is my final portfolio presentation for my CSC506 - Design and Analysis of Algorithms at Colorado State University Global course

Important Note

This presentation showcases the CSC506 Final Portfolio Presentation.

It focuses the integration of the prior milestone and critical thinking projects and on the last module of the course: Bubble Quickselect Sets; as well as the final comprehensive performance analysis.

Only the overview, analysis, and recommendation guides features of the Search Comparison, Linked Structures, Tree Map, Hash Priority, and Graph Algorithms modules are explored in this presentation.

2

The purpose of this slide is to explain the presentation scope.

Note that only all the Bubble “Quickselect Sets” module features are explored fully, as well as the final comprehensive performance analysis.

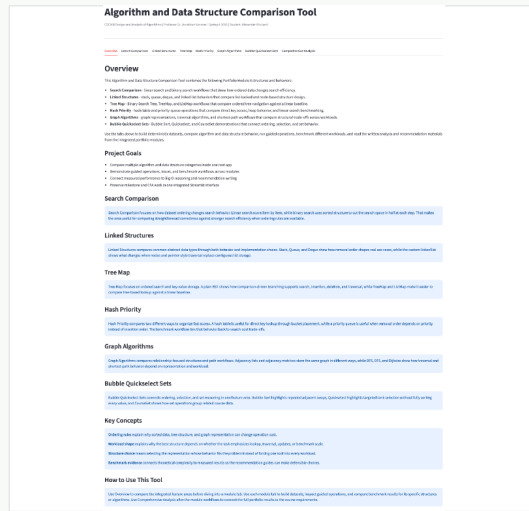
Only the overview, analysis, and recommendation guides features of the Search Comparison, Linked Structures, Tree Map, Hash Priority, and Graph Algorithms modules are explored in this presentation, as these modules were previously examined in detail in prior course assignments.

App Landing Page

Show the root application as the unified entry point for the complete Portfolio Module 8 system.

Key Takeaway

- One Streamlit entry point unifies every portfolio module.
- Tabs separate search, structures, maps, hashes, graphs, sets, and analysis.
- The screenshot proves the final system is integrated, not separate demos.



This is the Landing page of the application. It gives an overview of the application feature and provides navigation tabs to Search Comparison, Linked Structures, Tree Map, Hash Priority, Graph Algorithms, Bubble Quickselect Sets, and Comprehensive Analysis features.

It is important to note that each tab represents a different module of the project.

- Search Comparison focuses on comparing two fundamental search algorithms
- Linked Structures focuses on access.
- Tree Map focuses on ordered lookup.
- Hash Priority focuses on key-based retrieval and priority ordering.
- Graph Algorithms focuses on relationships, traversal, and shortest paths.
- Bubble Quickselect Sets focuses on uniqueness, sorting, and rank selection.
- Comprehensive Analysis is a performance and comparison analysis of algorithms and data structures explored in this project.

Search Comparison Overview

Introduce the Search Comparison module and its workflow for linear search, binary search, and supporting analysis tabs.

Algorithm and Data Structure Comparison Tool

Compare and contrast different algorithms and data structures using the Search Comparison module.

Home Search Comparison Dataset Builder Search Playground Compare Algorithms Benchmark Lab Big-O Analysis Recommendation Guide

Search Comparison

Overview

This module compares the performance of different search algorithms using the Search Comparison module. It provides a workflow for comparing different search algorithms and data structures using the Search Comparison module. The workflow includes the following steps:

- Dataset Builder: Prepare input values for the search algorithms.
- Search Playground: Run the search algorithms.
- Compare Algorithms: Compare the search algorithms side-by-side.
- Benchmark Lab: Measure the performance of the search algorithms.
- Big-O Analysis and Recommendation Guide: Explain the measured results.

Included Workflows

- Dataset Builder: Prepare input values for the search algorithms.
- Search Playground: Run the search algorithms.
- Compare Algorithms: Compare the search algorithms side-by-side.
- Benchmark Lab: Measure the performance of the search algorithms.
- Big-O Analysis and Recommendation Guide: Explain the measured results.

Project Overview

This module compares the performance of different search algorithms using the Search Comparison module. It provides a workflow for comparing different search algorithms and data structures using the Search Comparison module. The workflow includes the following steps:

- Dataset Builder: Prepare input values for the search algorithms.
- Search Playground: Run the search algorithms.
- Compare Algorithms: Compare the search algorithms side-by-side.
- Benchmark Lab: Measure the performance of the search algorithms.
- Big-O Analysis and Recommendation Guide: Explain the measured results.

Dataset Rules

All datasets in this project are used as input for the search algorithms. The datasets are used to compare the performance of different search algorithms. The datasets are used to compare the performance of different search algorithms. The datasets are used to compare the performance of different search algorithms.

Linear Search

Linear search is a simple search algorithm that finds the index of a target value in an array. It works by iterating over the array and comparing each element to the target value. If the target value is found, the index is returned. If the target value is not found, the function returns -1.

Binary Search

Binary search is a more efficient search algorithm that finds the index of a target value in a sorted array. It works by repeatedly dividing the array in half and comparing the middle element to the target value. If the target value is found, the index is returned. If the target value is not found, the function returns -1.

Key Concepts

Big-O notation is a way to describe the time complexity of an algorithm. It is used to compare the performance of different algorithms. Big-O notation is used to compare the performance of different algorithms. Big-O notation is used to compare the performance of different algorithms.

Key Takeaway

- Linear search and binary search are compared in one workflow.

- Local tabs move from setup to execution, benchmark, and guidance.

- The screen turns a textbook comparison into a visible lab.

4

This is the landing page of the “Search Comparison” feature, showcasing the functionality of the Search algorithms module. It provides an overview of the different search algorithms.

The Feature’s tabs:

- Overview introduces the concept.
- Dataset Builder prepares input values.
- Search Playground runs the algorithms.
- Compare Algorithms places them side by side.
- Benchmark Lab measures behavior.
- Big-O Analysis and Recommendation Guide explains what the measured results mean.

Search Comparison Big-O Analysis

Connect the search module to Big-O notation and explain why linear and logarithmic growth behave differently.

Key Takeaway

- Linear search can inspect every item: $O(n)$.
- Binary search halves the remaining range: $O(\log n)$.
- The screenshot connects Big-O notation to measured comparison behavior.

Algorithm and Data Structure Comparison Tool

Copyright © 2020 by the author(s). All rights reserved. This work is licensed under a Creative Commons Attribution 4.0 International License.

Search Comparison

Linear Search, Binary Search, Hash Table, B-Tree, AVL Tree, Red-Black Tree, Segment Tree, Fenwick Tree, Trie, Suffix Array, LCA, etc.

Big-O Analysis of Linear Search and Binary Search

This section provides a theoretical and empirical analysis of the time complexity of Linear Search and Binary Search. The analysis is based on the number of comparisons required to find an element in an array of size n .

Linear Search: In Linear Search, the algorithm iterates through each element of the array until it finds the target element. The worst-case time complexity is $O(n)$, where n is the number of elements in the array.

Binary Search: In Binary Search, the algorithm repeatedly divides the array into two halves until it finds the target element. The worst-case time complexity is $O(\log n)$, where $\log$ is the base-2 logarithm.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=1000$.

Search Method	Comparisons
Linear Search	1000
Binary Search	10

The results show that Binary Search is significantly faster than Linear Search, as it requires only 10 comparisons to find the target element, while Linear Search requires 1000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=10000$.

Search Method	Comparisons
Linear Search	10000
Binary Search	14

The results show that Binary Search is significantly faster than Linear Search, as it requires only 14 comparisons to find the target element, while Linear Search requires 10000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=100000$.

Search Method	Comparisons
Linear Search	100000
Binary Search	17

The results show that Binary Search is significantly faster than Linear Search, as it requires only 17 comparisons to find the target element, while Linear Search requires 100000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=1000000$.

Search Method	Comparisons
Linear Search	1000000
Binary Search	20

The results show that Binary Search is significantly faster than Linear Search, as it requires only 20 comparisons to find the target element, while Linear Search requires 1000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=10000000$.

Search Method	Comparisons
Linear Search	10000000
Binary Search	23

The results show that Binary Search is significantly faster than Linear Search, as it requires only 23 comparisons to find the target element, while Linear Search requires 10000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=100000000$.

Search Method	Comparisons
Linear Search	100000000
Binary Search	26

The results show that Binary Search is significantly faster than Linear Search, as it requires only 26 comparisons to find the target element, while Linear Search requires 100000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=1000000000$.

Search Method	Comparisons
Linear Search	1000000000
Binary Search	29

The results show that Binary Search is significantly faster than Linear Search, as it requires only 29 comparisons to find the target element, while Linear Search requires 1000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=10000000000$.

Search Method	Comparisons
Linear Search	10000000000
Binary Search	32

The results show that Binary Search is significantly faster than Linear Search, as it requires only 32 comparisons to find the target element, while Linear Search requires 10000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=100000000000$.

Search Method	Comparisons
Linear Search	100000000000
Binary Search	35

The results show that Binary Search is significantly faster than Linear Search, as it requires only 35 comparisons to find the target element, while Linear Search requires 100000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=1000000000000$.

Search Method	Comparisons
Linear Search	1000000000000
Binary Search	38

The results show that Binary Search is significantly faster than Linear Search, as it requires only 38 comparisons to find the target element, while Linear Search requires 1000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=10000000000000$.

Search Method	Comparisons
Linear Search	10000000000000
Binary Search	41

The results show that Binary Search is significantly faster than Linear Search, as it requires only 41 comparisons to find the target element, while Linear Search requires 10000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=100000000000000$.

Search Method	Comparisons
Linear Search	100000000000000
Binary Search	44

The results show that Binary Search is significantly faster than Linear Search, as it requires only 44 comparisons to find the target element, while Linear Search requires 100000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=1000000000000000$.

Search Method	Comparisons
Linear Search	1000000000000000
Binary Search	47

The results show that Binary Search is significantly faster than Linear Search, as it requires only 47 comparisons to find the target element, while Linear Search requires 1000000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=10000000000000000$.

Search Method	Comparisons
Linear Search	10000000000000000
Binary Search	50

The results show that Binary Search is significantly faster than Linear Search, as it requires only 50 comparisons to find the target element, while Linear Search requires 10000000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=100000000000000000$.

Search Method	Comparisons
Linear Search	100000000000000000
Binary Search	53

The results show that Binary Search is significantly faster than Linear Search, as it requires only 53 comparisons to find the target element, while Linear Search requires 100000000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=1000000000000000000$.

Search Method	Comparisons
Linear Search	1000000000000000000
Binary Search	56

The results show that Binary Search is significantly faster than Linear Search, as it requires only 56 comparisons to find the target element, while Linear Search requires 1000000000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=10000000000000000000$.

Search Method	Comparisons
Linear Search	10000000000000000000
Binary Search	59

The results show that Binary Search is significantly faster than Linear Search, as it requires only 59 comparisons to find the target element, while Linear Search requires 10000000000000000000 comparisons.

The following table shows the measured comparison counts for Linear Search and Binary Search on an array of size $n=100000000000000000000$.

Search Method	Comparisons
Linear Search	100000000000000000000
Binary Search	62

The results show that Binary Search is significantly faster than Linear Search, as it requires only 62 comparisons to find the target element, while Linear Search requires 100000000000000000000 comparisons.

This showcases the “Search Comparison Big-O Analysis” for the “Search Comparison” feature.

The analysis uses a Big-O perspective to analyze the search algorithms.

The analysis results:

- Linear search shows worst-case $O(n)$ behavior because it may need to inspect every value in the dataset.
- Binary search has worst-case $O(\log n)$ behavior because each comparison removes one half of the remaining interval.

Note that the analysis implemented a theoretical and empirical approach.

The analysis compares counts, step traces, and benchmark evidence.

Translate the search comparison evidence into practical recommendations about when to use each algorithm.

Algorithm and Data Structure Comparison Tool

Discover the strengths and weaknesses of various algorithms and data structures, helping you choose the best one for your specific use case.

Discover

Search Complexity

Linked Structures

Tree Traversal

Hash Tables

Graph Algorithms

Sorting Algorithms

Dynamic Programming

Combinatorics

Maths

Search Comparison

Discover

Search Complexity

Search Algorithms

Complexity Analysis

Performance Metrics

Real-world Scenarios

Recommendation Guide

Recommendation Guide: When to Choose Each Search Algorithm

Overview

Choosing the right search algorithm depends on the data structure, the size of the dataset, and whether the data is already sorted or not. This guide provides a checklist to help you decide between linear search and binary search.

When to Use Linear Search

Linear search is the best choice when:

- The dataset is small (less than 1000 elements). It's simple to implement and works well for unsorted data.
- The data is unsorted and you only need to find one or a few items. Sorting the data first takes O(n log n) time, which is worse than the O(n) time of a single linear search.
- The data structure does not support random access. Linear search requires jumping to different parts of the data (e.g., linked lists, trees, and graph-based data structures) which might require sequential access, making linear search the only option.
- The data changes frequently. Maintaining a sorted list requires extra work, so keeping the data unsorted is simpler.
- You need to find all occurrences. Linear search naturally visits every element, so collecting all matches requires no extra logic.

When to Use Binary Search

Binary search is the better choice when:

- The dataset is large and already sorted or can be sorted. Binary search's O(log n) performance makes it a great choice for large datasets. For example, searching 1,000,000 sorted elements requires at most 20 comparisons, while linear search would require 1,000,000 comparisons.
- The data is sorted and you need to find one or a few items. Binary search is much faster than linear search, especially for large datasets. The time complexity of binary search is O(log n), which is much better than the O(n) time of linear search.
- The data is stored in a random access data structure (e.g., arrays, linked lists, B-trees, etc.). Binary search requires random access to the data, which is not possible for linked lists or trees.
- The data is sorted and you need to find all occurrences. Binary search can be used to find all occurrences of a value in a sorted array.

Key Takeaway

- Binary search wins when sorted data or repeated lookup is available.
- Linear search still fits small, unsorted, or changing datasets.
- The guide frames algorithm choice as workload fit.

This showcases the recommendation guide for the “Search Comparison” feature.

The recommendation guide uses analysis results to guide decision-making:

- Binary search is the stronger choice when the dataset is pre-sorted or when repeated searches are required.
- Linear search is a practical choice when the dataset is small, unsorted, or changing often.

The key lesson from the guide and analysis is that algorithm choice depends on workload rather than on a universal ranking.

Linked Structures Overview

Introduce the *Linked Structures* module and its workflows for *Stack*, *Queue*, *Deque*, and *LinkedList* behavior.

Algorithm and Data Structure Comparison Tool

©2020 Google and Microsoft Algorithm Comparison Tool. All rights reserved. Contact: [support@algorithmcomparison.com](#)

Home | **Linked Structures** | The Map | Data Types | Stack Algorithms | Queue Algorithms | Deque Algorithms | LinkedList Algorithms

Linked Structures

Home | **Linked Structures** | The Map | Data Types | Stack Algorithms | Queue Algorithms | Deque Algorithms | LinkedList Algorithms

Overview

This Algorithm and Data Structure Comparison Tool compares the following major data structures:

- Stacks - LIFO (Last In First Out) structure with push, pop, and peek.
- Queues - FIFO (First In First Out) structure with enqueue, dequeue, and peek.
- Deque - Double-Ended Queue structure with push, pop, enqueue, dequeue, and peek.
- LinkedList - A sequence of nodes, each containing data and a pointer to the next node.

Use this tool to build datasets, run individual data structure operations, compare their time complexity, benchmark them, and read the written analysis and recommendation guide.

Stack

Stack is a LIFO (Last In First Out) data structure. The most recently added item is the first one removed. This module is implemented with a Python 3.6.8 version, and uses all other data types of the structure and not a linked list.

Queue

Queue is a FIFO (First In First Out) data structure. The earliest item added is the first one removed. This module is implemented with a Python 3.6.8 version, and uses all other data types of the structure and not a linked list.

Deque

Deque is a Double-Ended Queue structure. It allows adding and removing items from both the front and rear. This module is implemented with a Python 3.6.8 version, and uses all other data types of the structure and not a linked list.

LinkedList

LinkedList is a sequence of nodes, each containing data and a pointer to the next node. This module is implemented with a Python 3.6.8 version, and uses all other data types of the structure and not a linked list.

Key Concepts

Linked list is a linear data structure. It consists of a sequence of nodes, each containing data and a pointer to the next node. This module is implemented with a Python 3.6.8 version, and uses all other data types of the structure and not a linked list.

How to Use This Tool

This tool is designed to be used by anyone who is interested in learning about the behavior of different data structures. It provides a user-friendly interface for building datasets, running operations, comparing structure behavior, benchmarking workloads, and reading supporting analysis.

Key Takeaway

- Stack, Queue, Deque, and LinkedList each model a different access pattern.
- The module links operations, benchmarks, and written analysis.
- The screen shows why similar structures can behave differently.

This is the landing page of the “Linked Structures” feature, showcasing the functionality of the Linked structures module. It provides an overview of the different abstract data types.

The data structures explored are list-backed Stack, Queue, and Deque, along with a custom doubly linked `LinkedList`.

The feature provides the user with the following tabs/functionality: build datasets, run operations, compare structure behavior, benchmark workloads, and read supporting analysis.

The feature functionality allows the user to explore LIFO, FIFO, double-ended access, and node traversal concepts and implement their behavior. In practical terms, this module explains why linear structures can behave very differently under different workloads.

Linked Structures Written Analysis

Explain how the written analysis connects structure behavior to shifting, endpoint work, and traversal costs.

Key Takeaway

- Implementation details shape real performance, not just ADT names.

- Endpoint work, shifting, and traversal carry different costs.

- Benchmark evidence shows each structure has a best-fit workload.

Algorithm and Data Structure Comparison Tool

https://github.com/algorithm-visualizer/algorithm-visualizer

Linked Structures

Written Analysis

Data Structure Performance Analysis

Overview

Linked structures are a type of data structure that store data in a sequence of nodes. Each node contains a pointer to the next node in the sequence. This structure is used to store data in a way that allows for efficient traversal and insertion/deletion of elements.

The analysis of linked structures focuses on understanding the performance of different operations (insertion, deletion, traversal) and how these operations are affected by the underlying implementation details (e.g., node structure, pointer management).

The analysis also considers the trade-offs between different data structures (e.g., arrays, linked lists, hash tables) and how these trade-offs are influenced by the specific requirements of the application.

The analysis aims to provide a comprehensive overview of the performance characteristics of linked structures and to identify the factors that influence their performance.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

The analysis also includes a discussion of the various applications of linked structures and the challenges associated with their use.

The analysis concludes with a summary of the key findings and a discussion of the implications for the design and implementation of data structures.

This showcases the “Linked Structures Written Analysis” for the “Linked Structures” feature.

The analysis results show that implementation details matter just as much as abstract definitions of the data structure.

For example:

- A stack performs well when push and pop happen on the favorable end of a Python list.
- Queue and Dequeue behavior depends on which end receives most of the work.
- A linked list avoids some shifting costs, but traversal still grows with the structure length.

Linked Structures Recommendation Guide

Show how the module converts structure tradeoffs into workload-based recommendations.

Algorithm and Data Structure Comparison Tool
Comparing and contrasting algorithms and data structures to choose the best one for your application.

Search: [Linked Structures](#) [Trees](#) [Hash Tables](#) [Sorting Algorithms](#) [Graphs](#) [Dynamic Programming](#)

Linked Structures

Sort by: [Relevance](#) [Alphabetical](#) [Complexity](#) [Difficulty](#) [Popularity](#) [Newest](#) [Oldest](#)

Recommendation Guide

Recommendation Guide: Choosing the Right Data Structure

Overview

Choosing the right data structure is a key part of writing efficient code. This guide provides a high-level overview of the tradeoffs between different data structures and how to choose the right one for your application. The guide is organized into sections that cover different types of data structures and their use cases. The goal is to help you understand the strengths and weaknesses of each structure and make informed decisions about which one to use in your code.

When to Use Stack

Stacks are used to store data in a last-in, first-out (LIFO) manner. They are useful for a variety of applications, including:

- 1. **Function calls:** Stacks are used to store the return addresses of function calls. When a function is called, its return address is pushed onto the stack. When the function returns, the return address is popped from the stack, and control is transferred back to the caller.
- 2. **Undo/redo operations:** Stacks are used to store the state of an application before and after a change. When a change is made, the state is pushed onto the stack. When the user wants to undo the change, the state is popped from the stack, and the application is restored to its previous state.
- 3. **Expression evaluation:** Stacks are used to store the operators and operands of an expression. When an operator is encountered, the operators are popped from the stack, and the expression is evaluated. When an operand is encountered, the operand is pushed onto the stack.
- 4. **Depth-first search:** Stacks are used to store the nodes of a graph that have been visited. When a node is visited, it is pushed onto the stack. When the search reaches a dead end, the node is popped from the stack, and the search continues with the next node.

Stacks are also used in many other applications, such as parsing, text processing, and game AI.

When to Use Queue

Queues are used to store data in a first-in, first-out (FIFO) manner. They are useful for a variety of applications, including:

- 1. **Task scheduling:** Queues are used to store the tasks that need to be executed. When a task is added, it is pushed onto the queue. When the scheduler is ready to execute a task, it is popped from the queue, and the task is executed.
- 2. **Buffering:** Queues are used to store data that is being transferred between two systems. When data is received, it is pushed onto the queue. When the destination system is ready to receive the data, it is popped from the queue.
- 3. **Simulation:** Queues are used to store the events that need to be processed in a simulation. When an event is added, it is pushed onto the queue. When the simulation is ready to process an event, it is popped from the queue.
- 4. **Network routing:** Queues are used to store the packets that need to be routed. When a packet is added, it is pushed onto the queue. When the router is ready to route a packet, it is popped from the queue.

Queues are also used in many other applications, such as database management, operating systems, and network protocols.

When to Use Deque

Dequeues are used to store data in a first-in, first-out (FIFO) or last-in, first-out (LIFO) manner. They are useful for a variety of applications, including:

- 1. **Task scheduling:** Dequeues are used to store the tasks that need to be executed. When a task is added, it is pushed onto the deque. When the scheduler is ready to execute a task, it is popped from the deque.
- 2. **Buffering:** Dequeues are used to store data that is being transferred between two systems. When data is received, it is pushed onto the deque. When the destination system is ready to receive the data, it is popped from the deque.
- 3. **Simulation:** Dequeues are used to store the events that need to be processed in a simulation. When an event is added, it is pushed onto the deque. When the simulation is ready to process an event, it is popped from the deque.
- 4. **Network routing:** Dequeues are used to store the packets that need to be routed. When a packet is added, it is pushed onto the deque. When the router is ready to route a packet, it is popped from the deque.

Dequeues are also used in many other applications, such as database management, operating systems, and network protocols.

Key Takeaway

- Stack supports LIFO; Queue supports FIFO.

- Deque helps when both ends matter.

- LinkedList fits node-based insertion, deletion, and traversal cases.

This showcases the recommendation guide for the “Linked Structures” feature.

The recommendation guide explains how to match a structure to the operation pattern of an application.

For example:

- A stack is best suited for last-in, first-out behavior.
- A queue is best suited for first-in, first-out workflows.
- A deque is best when both ends matter.
- The `LinkedList` is best when node-based insertion, removal, and traversal behavior fit the problem better than list shifting.

Tree Map Overview

Introduce the *Tree Map* module and its comparison between *BST*, *TreeMap*, and *ListMap* behavior.

Key Takeaway

- BST, *TreeMap*, and *ListMap* support ordered key workflows.
- Insertion order changes tree shape and practical performance.
- The screen introduces lookup, traversal, height, and balance evidence.

Algorithm and Data Structure Comparison Tool

Home | Overview | Tree Map | Binary Search Tree | ListMap | Complexity Overview | Key Concepts

Tree Map

Overview

The *Tree Map* module provides a comprehensive overview of the importance of tree maps in computer science. It combines key-based lookup with an ordered structure. The feature explores Binary Search Tree, a BST-backed *TreeMap*, and a list-backed *ListMap* baseline. It also explores insertion, search, deletion, traversal, minimum and maximum lookups, height reporting, balance detection operations, and a map comparison workflow.

Project Goals

- Provide a clear, concise, and accessible overview of the importance of tree maps in computer science.
- Explore the differences between *BST*, *TreeMap*, and *ListMap* behavior.
- Introduce the concepts of lookup, traversal, height, and balance evidence.

Binary Search Tree

A Binary Search Tree (BST) is a type of tree structure where each node has at most two children, referred to as the left child and the right child. The left child must have a key less than the parent node's key, and the right child must have a key greater than the parent node's key.

TreeMap

TreeMap is a Java class that implements the *Map* interface. It uses a Red-Black tree to store the entries in the map. This ensures that the entries are stored in a sorted order based on their keys.

ListMap

ListMap is a Java class that implements the *Map* interface. It uses a linked list to store the entries in the map. This ensures that the entries are stored in the order they were inserted into the map.

Complexity Overview

Operation	<i>BST</i>	<i>TreeMap</i>	<i>ListMap</i>
Lookup	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$	$O(n)$
Height	$O(\log n)$	$O(\log n)$	$O(n)$
Balance	$O(\log n)$	$O(\log n)$	$O(n)$

Key Concepts

Key Concepts include: Binary Search Tree, *TreeMap*, *ListMap*, lookup, traversal, height, and balance evidence.

How to Use This Tool

This tool is designed to help you understand the differences between *BST*, *TreeMap*, and *ListMap* behavior. It provides a clear, concise, and accessible overview of the importance of tree maps in computer science. It also provides a detailed comparison of the complexity of various operations for each data structure.

10

This is the landing page of the “Tree Map” feature, showcasing the functionality of the tree map module. It provides an overview of the importance of tree maps in computer science. They combine key-based lookup with an ordered structure. The feature explores Binary Search Tree, a BST-backed *TreeMap*, and a list-backed *ListMap* baseline. It also explores insertion, search, deletion, traversal, minimum and maximum lookups, height reporting, balance detection operations, and a map comparison workflow.

The features allow users to explore how a tree shape influences performance.

It shows that tree-backed lookup can be efficient when the tree height is low; it shows how a plain BST is not self-correcting.

The feature also provides an explanation of why ordered lookup can be powerful and why insertion order still matters.

- The analysis ties tree shape to benchmark results.

The observed increase in system loads is a good indicator of system stress. The increase in measures when system loads are increased is enough from a health perspective to require system operators to adjust design. This behavior indicates that system loads are important in their evaluation of system health, and suggest that the relation of another important design (CMI) and the

Note: the same structure can behave very differently depending on insertion order.

Tree Map Recommendation Guide

Show when tree-backed maps are a strong fit and when simpler linear maps remain practical.

Algorithm and Data Structure Comparison Tool

Overview

Recommendation Guide

Recommendation Guide: Choosing Between TreeMap and ListMap

Overview

TreeMap and ListMap are both backed by ordered data structures, but they differ in their underlying implementation. TreeMap is backed by a red-black tree, while ListMap is backed by a linked list. This guide provides a comparison of the two data structures and offers recommendations on when to use each.

When to Use TreeMap

- TreeMap is a good choice when you need a map that is ordered by its keys. It is also a good choice when you need a map that is backed by a red-black tree.
- TreeMap is a good choice when you need a map that is backed by a red-black tree and you need a map that is ordered by its keys.
- TreeMap is a good choice when you need a map that is backed by a red-black tree and you need a map that is ordered by its keys.

When to Use ListMap Instead of TreeMap

- ListMap is a good choice when you need a map that is ordered by its keys and you need a map that is backed by a linked list.
- ListMap is a good choice when you need a map that is ordered by its keys and you need a map that is backed by a linked list.
- ListMap is a good choice when you need a map that is ordered by its keys and you need a map that is backed by a linked list.

When a Plain BST-Based Map is a Strong Choice

- A plain BST-based map is a good choice when you need a map that is ordered by its keys and you need a map that is backed by a plain BST.
- A plain BST-based map is a good choice when you need a map that is ordered by its keys and you need a map that is backed by a plain BST.
- A plain BST-based map is a good choice when you need a map that is ordered by its keys and you need a map that is backed by a plain BST.

Key Takeaway

- TreeMap fits ordered traversal and repeated lookup.

- ListMap remains practical for small or simple workloads.

- The guide warns that skew can erase the tree advantage.

This showcases the recommendation guide for the “Tree Map” feature.

The recommendation guide explains that:

- `TreeMap` is best suited for situations when ordered traversal, repeated lookup, and minimum or maximum queries are required.
- `ListMap` is easier to implement, but less efficient for large datasets, but it can still be a practical implementation for modest-sized datasets or simpler workloads.

Tree-backed maps are useful when the tree shapes are not skewed.

- Hash tables optimize direct key lookup.
- Priority queues optimize repeated highest or lowest priority retrieval.
- The module compares two different fast-access problems.

13

The module compares two solutions for two different access requirements: that is, hash tables are built for direct key lookup, while heaps are designed for repeated access to the next highest or lowest priority item.

Hash Priority Written Analysis

Explain how the written analysis connects collision behavior, heap maintenance, and operation cost.

Algorithm and Data Structure Comparison Tool

Hash Priority

Written Analysis

Data Structure Performance Analysis

Overview

Hash tables are a fundamental data structure used for efficient storage and retrieval of data. They are implemented using an array of buckets, where each bucket stores a list of elements that hash to the same value. The performance of a hash table depends on the quality of the hash function and the distribution of the keys. A good hash function should distribute keys uniformly across the buckets, minimizing collisions. Collisions occur when two different keys hash to the same bucket, which can increase the time complexity of operations. Separate chaining is a common collision resolution technique, where each bucket contains a linked list of elements. This ensures that operations remain efficient even in the presence of collisions. The time complexity for insert, search, and delete operations is $O(1)$ on average, assuming a good hash function and uniform distribution of keys. However, in the worst case, where all keys hash to the same bucket, the complexity can degrade to $O(n)$.

Hash Table

Hash tables are a fundamental data structure used for efficient storage and retrieval of data. They are implemented using an array of buckets, where each bucket stores a list of elements that hash to the same value. The performance of a hash table depends on the quality of the hash function and the distribution of the keys. A good hash function should distribute keys uniformly across the buckets, minimizing collisions. Collisions occur when two different keys hash to the same bucket, which can increase the time complexity of operations. Separate chaining is a common collision resolution technique, where each bucket contains a linked list of elements. This ensures that operations remain efficient even in the presence of collisions. The time complexity for insert, search, and delete operations is $O(1)$ on average, assuming a good hash function and uniform distribution of keys. However, in the worst case, where all keys hash to the same bucket, the complexity can degrade to $O(n)$.

Hash Function Choice and Collision Resolution

Hash functions are used to map keys to a fixed-size integer value, which is then used to index into the array of buckets. The choice of hash function is critical to the performance of the hash table. A good hash function should distribute keys uniformly across the buckets, minimizing collisions. Common hash functions include the division method, the multiplication method, and the Fibonacci hashing method. Collision resolution techniques are used to handle collisions, which occur when two different keys hash to the same bucket. Separate chaining is a common technique, where each bucket contains a linked list of elements. Other techniques include open addressing and double hashing.

Priority Queue

Priority queues are a data structure that stores elements in a way that allows for efficient retrieval of the element with the highest priority. They are implemented using a binary heap, which is a complete binary tree where the root node is the highest priority element. The time complexity for insert and extract operations is $O(\log n)$, while the time complexity for peek operations is $O(1)$. The heap structure ensures that the highest priority element is always at the root, making it easy to retrieve. The binary heap is a self-adjusting structure, meaning that it can maintain its heap property after insert and extract operations.

Key Takeaway

- Healthy hash tables support average $O(1)$ key operations.

- Collisions still affect bucket-level search cost.

- Heap operations preserve priority order with $O(\log n)$ maintenance.

14

This showcases the “Hash Priority Written Analysis” for the “Hash Priority” feature.

The analysis uses a Big-O perspective to analyze hash tables and priority queues [from both theoretical and practical viewpoints.

The analysis results:

- A hash table can have a complexity average of $O(1)$ for insert, search, and delete operations because keys map directly to bucket locations. It is important to note, however, that collisions may be an issue. Separate chaining preserves correctness, but depending on the key distribution, searching inside a bucket could be more expensive.
- The priority queue shows different results. Heap insert and extract operations are typically $O(\log n)$ because the heap may need to restore order. Peek remains efficient because the highest or lowest priority value is stored at the root.

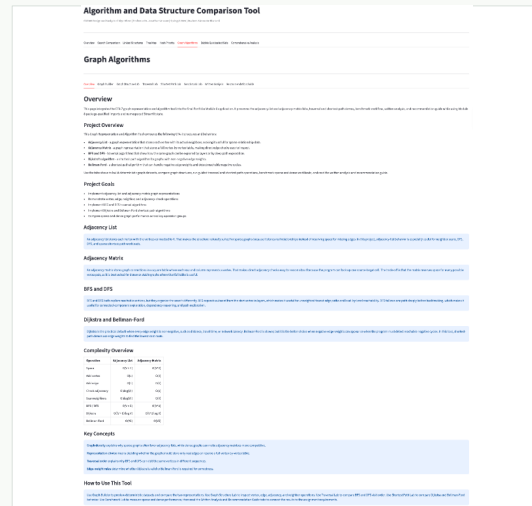
- Hash tables are the best choice when repeated key-based lookups are required and when key distribution can be maintained.
- Priority queues are the best choice when the application requires repeatedly removing the highest or lowest priority value.
- Linear search is useful for simple implementation, educational purposes, and as a comparison point.

Graph Algorithms Overview

Introduce the Graph Algorithms module and its workflows for representation, traversal, shortest paths, and benchmarking.

Key Takeaway

- Graphs model relationships between objects.
- The module compares adjacency lists, matrices, traversal, and paths.
- Benchmarks separate sparse and dense graph behavior.



This is the landing page of the “Graph Algorithms” feature.

It highlights the importance of graphs, which are used to represent relationships between objects. The module showcases adjacency-list and adjacency-matrix data structure representations, and BFS, DFS, Dijkstra, and Bellman-Ford algorithms. It also provides labs for sparse and dense graph behavior.

The features tabs:

- Graph Builder allows the user to build different graph structures.
- Graph Structure Lab showcases the different graph behaviors.
- Traversal Lab showcases BFS and DFS.
- Shortest Path Lab compares Dijkstra and Bellman-Ford algorithms for short path operations.
- Benchmark Lab runs benchmarks for the different graph structures and the algorithms.
- Written Analysis and Recommendation Guide analyzes the lab results and base on the results, gives guidance.

Algorithm and Data Structure Comparison Tool

[Home](#)
[Search](#)
[Contact Us](#)
[Privacy Policy](#)
[Terms of Service](#)
[Feedback](#)
[Help](#)
[About Us](#)
[Sitemap](#)

Graph Algorithms

Overview | Search Results | Sign & Share Your Lab | Privacy Policy | Contact Us | Feedback | Researcher Tools | Researcher Profile | Researcher Dashboard

Written Analysis

Graph Representation and Algorithm Performance Analysis

Overview

Graphs are important in computer science because they model relationships between objects. A graph can describe roads between cities, links between network devices, dependencies between tasks, or relationships between people. In this project, two graph storage models are implemented and evaluated: [Adjacency List \(AL\)](https://arxiv.org/abs/1603.04462) and [Adjacency Matrix \(AM\)](https://arxiv.org/abs/1603.04462). The project also evaluates four graph algorithms: breadth first search (BFS), depth first search (DFS), Dijkstra's shortest path algorithm, and Bellman-Ford.

The purpose of this analysis is to examine how each representation behaves both theoretically and practically. Theoretical analysis uses Big O notation to describe how performance grows as the number of vertices n and edges m increases. Practical analysis uses benchmarks collected from the BenchmarkLib across sparse and dense graph networks using vertex counts of 10, 30 and 100. The used benchmark data comes from <https://github.com/andrewyeh/BenchmarkLib> and is available at <https://www.kaggle.com/andrewyeh/benchmarklib>.

It is important to note that representation choice depends strongly on the workload. The link-based model works out neighbor relationships, which makes it a good fit for sparse workloads. The matrix-based model allocates a row-column position for each source-target pair, thus designating in advance, matrix positions where direct edge checks are common (Xie (2014), e.g.; Gready & Hefley, 2008). The benchmark scenario

This project suggests that point density is a poor L2L3 metric. It also shows how strongly graph density influences the practice difference between the based and matrix based strength.

Adjacency List Representation

`adjacency_list` stores each vertex inside the vertex directly connected to it. In this project, the list representation was dictionary-based neighbor maps, which stores a vertex only stores relationships that actually exist. This structure fits sparse graphs because absent edges do not require a stored source-target link.

From a Big-O perspective, list-based graph storage uses $O(V + E)$ space. Threshold algorithms without BFS and DFS also fit naturally with this representation because each step uses the current vertex's stored neighbors. In a sparse network, that behavior avoids scanning many empty relationships. This is why list-based representations are commonly used in real world where edge density remains much lower than the maximum number of possible vertex pairs [Cormen et al., 2009].

In practice, the saved benchmarks result strongly favor list-based storage on sparse workloads. *IR* *version* in the sparse workload, for list implementation is faster than build, neighbor scan, DFS, BFS, Dijkstra, Bellman-Ford, and edge removal. The only sparse *IR* *version* expenditure where the matrix wins is direct adjacency checking, and even there the matrix advantage is modest. These results show that list-based storage

is the strongest default for the kind of sparse relationship data that *spyzoo* offers in real applications.

Adjacency Matrix Representation

[AdjacencyMatrix\[graph\]](#) shows graph connections in a square table. Each row and column represents a vertex, and each cell records whether a source-target pair has an edge. If the row and column are already known, an adjacency check can be performed in $O(1)$ time. That direct lookup is the main theoretical advantage of the matrix representation (CSE Global, n.d.; Joly et al., 2020).

The trade-off is space. A matrix uses [O\(N²\)](#) storage because it must allocate a cell for each source-target pair, even when most pairs do not have edges. This makes matrix storage less attractive for sparse weighted edges, more reasonable in dense workloads because a large share of the allocated cells contain useful edge information.

Traversal and Shortest-Path Algorithms

DFS and BFS both explore each *subtree* of nodes, but they organize the search in different ways. BFS explores outward from the start vertex in layers, while DFS follows one path as deeply as before it backtracks (Kortbeek, State University, Global, 2008; Lintyas & Widi, 2008). In this project, the Traveling Lab shows that the same graph can produce different visit orders depending on whether the program uses DFS or BFS.

Key Takeaway

- Shortest-path algorithms depend on edge-weight assumptions.

- Dijkstra is faster, but it is best suited for a graph with non-negative edge weights.
- Bellman-Ford is slower but supports negative edges and negative-cycle detection.

Graph Algorithms Recommendation Guide

Show how the graph module turns representation and algorithm tradeoffs into workload-based recommendations.

Key Takeaway

- BFS fits breadth-oriented unweighted exploration.
- DFS fits depth-oriented traversal.
- Dijkstra and Bellman-Ford serve different weighted-path constraints.

Algorithm and Data Structure Comparison Tool

Comparison tool for graph algorithms and data structures.

Overview: Graph Algorithms, Graph Data Structures, Graph Traversal, Graph Search, Graph Shortest Paths, Graph Connectivity, Graph Clustering, Graph Visualization.

Graph Algorithms

Overview: Graph Algorithms, Graph Data Structures, Graph Traversal, Graph Search, Graph Shortest Paths, Graph Connectivity, Graph Clustering, Graph Visualization.

Recommendation Guide: Choosing Graph Representations and Algorithms

Overview

Choosing the right graph representation and algorithm is critical for performance. This guide provides a systematic approach to selecting the best combination based on your workload characteristics.

When to Use an Adjacency List

1. The graph is sparse (low edge density).
2. The graph is undirected or the edges are symmetric.
3. The graph has a high degree of connectivity (many edges per node).
4. The graph is dynamic (edges are added or removed frequently).
5. The graph is used for shortest path algorithms (e.g., Dijkstra's, Bellman-Ford).
6. The graph is used for graph traversal (e.g., BFS, DFS).
7. The graph is used for graph clustering (e.g., community detection).
8. The graph is used for graph visualization (e.g., network diagrams).

When to Use an Adjacency Matrix Instead of an Adjacency List

1. The graph is dense (high edge density).
2. The graph is directed (edges are not symmetric).
3. The graph has a low degree of connectivity (few edges per node).
4. The graph is static (edges are not added or removed frequently).
5. The graph is used for shortest path algorithms (e.g., Dijkstra's, Bellman-Ford).
6. The graph is used for graph traversal (e.g., BFS, DFS).
7. The graph is used for graph clustering (e.g., community detection).
8. The graph is used for graph visualization (e.g., network diagrams).

When an Adjacency List is the Stronger Default Choice

1. The graph is sparse (low edge density).
2. The graph is undirected or the edges are symmetric.
3. The graph has a high degree of connectivity (many edges per node).
4. The graph is dynamic (edges are added or removed frequently).
5. The graph is used for shortest path algorithms (e.g., Dijkstra's, Bellman-Ford).
6. The graph is used for graph traversal (e.g., BFS, DFS).
7. The graph is used for graph clustering (e.g., community detection).
8. The graph is used for graph visualization (e.g., network diagrams).

18

This showcases the recommendation guide for the “Graph Algorithms” feature.

It explains when to use each graph structure and algorithm.

For example:

- Adjacency lists are useful when the graph is sparse or when neighbor scans are common.
- Adjacency matrices become more attractive when direct pair checks matter or when the graph is dense and stable.
- BFS is well-suited for breadth-oriented unweighted exploration.
- DFS is well-suited for depth-oriented traversal.
- Dijkstra is well-suited for non-negative weighted shortest paths.
- Bellman-Ford is well-suited to situations where negative weights are required.

Bubble Quickselect Sets Overview

Introduce the Module 8-native feature area for sets, Bubble Sort, Quickselect, benchmarks, and written analysis.

Key Takeaway

- Module 8 adds sets, Bubble Sort, and Quickselect.
- Sets answer membership; sorting answers full order.
- Quickselect answers one rank without fully sorting everything.

Algorithm and Data Structure Comparison Tool

©2016 Design and Analysis of Algorithms/Professor Dr. Jonathan Harvey (Spring 2016). Student: Alexander Ritschel

Overview Search Comparison Linked Structures Tree Traps Graph Algorithms **Bubble Quickselect Sets** Comparison Analysis

Bubble Quickselect Sets

Overview Bubble Sort Lab Bubble Sort Lab Quickselect Lab Benchmark Lab Written Analysis

Overview

This page demonstrates the Module 8-native implementations inside the single-Portfolio Module 8 Semester app: a Set ADT, Bubble Sort visualization, Quickselect kth smallest selection, benchmark comparison, and written analysis.

The tabs cover concepts overview, static and dynamic set behavior, adjacent comparison sorting, partition-based selection, repeatable benchmarks, and the MarkDown analysis source used for final portfolio discussion.

Feature Tabs

Tab	Purpose
Overview	Overview on the Bubble Sort and Quickselect concepts.
Bubble Sort Lab	Build sets and compare static operations with dynamic mutations.
Bubble Sort Lab	Run Bubble Sort and inspect adjacent comparison traces.
Quickselect Lab	Find kth-smallest values and inspect partition traces.
Benchmark Lab	Compare Bubble Sort, Quickselect, Quickselect, and Set rank selection.
Written Analysis	Read the MarkDown explanation and presentation recommendations.

Concept Overview

Sets store unique hashable values. Membership is the central operation, and static operations create derived sets without changing either original operand.

Bubble Sort repeatedly compares adjacent values and swaps out of order pairs. Its early exit optimization makes already sorted inputs finish after one no-swap pass.

Quickselect partitions values around a pivot to find one rank without pivoting for a complete sorted order.

Big-O Table

Concept	Best	Average	Worst	Space	Notes
Bubble Sort with early exit	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Traces every adjacent comparison on small inputs
Quickselect	$O(n)$	$O(n)$	$O(n^2)$	$O(n)$	Partial partitions data instead of fully sorting
Set membership	$O(1)$	average $O(1)$	average $O(1)$	$O(n)$	Hash-based membership
Set union/difference	$O(n + m)$	$O(n + m)$	$O(n + m)$	$O(n + m)$	Static operations return a new set

19

This is the landing page of the “Bubble Quickselect Sets” feature of the Bubble Quickselect Sets module. It includes an overview of the `CourseSet` implementation, the Bubble Sort routine with trace output, Quickselect for kth-smallest selection, benchmark comparison, and written analysis.

Build Set Lab: Initial Sets

Show the initial Set A and Set B workflow used to prepare the set demonstrations.

Algorithm and Data Structure Comparison Tool

Home | Bubble Quickselect Sets | Manual Operation Playground | Automatic Set Operations Demo | Set A contains

Bubble Quickselect Sets

Home | Bubble Quickselect Sets | Manual Operation Playground | Automatic Set Operations Demo | Set A contains

Build Set Lab

Set A: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Set B: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Manual Operation Playground

Set A: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Set B: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Current Set State

Set	Items	Count	Sum
A	1, 2, 3, 4, 5, 6, 7, 8, 9, 10	10	55
B	1, 2, 3, 4, 5, 6, 7, 8, 9, 10	10	55

Manual Operation History

1. Set A contains 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Automatic Set Operations Demo

Set A contains 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Automatic Demo Results

Set A contains 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Operation	Result	Time	Complexity
Set A contains 1, 2, 3, 4, 5, 6, 7, 8, 9, 10	True	0.0011 ms	O(1) average

Key Takeaway

- Set A and Set B establish the demonstration data.
- Uniqueness matters more than item position.
- The screen prepares the static and dynamic set operations.

20

In the Build Set Lab section of the feature, the user can build Set A and Set B, which are displayed so the user.

It is important to note that the central idea of sets is uniqueness. Duplicates are not allowed, and item membership to a set matters more than order.

Build Set Lab: Dynamic Operations

Demonstrate dynamic set operations such as *add*, *remove*, and *membership checks* on a selected target set.

Key Takeaway

- Dynamic operations maintain or inspect one target set.
- Add, remove, and contains focus on membership behavior.
- Hash-backed sets support efficient average-case checks.

Operation	Input	Time	Complexity
Set A Add	False, 5 + 5	0.0004 ms	$O(1)$ average
Set A Remove	True, 5 + 4	0.0010 ms	$O(1)$ average
Set A Union Set B	[1, 2, 3, 5, 8, 4, 9], 5 + 5	0.0030 ms	$O(n + m)$
Set A Intersection Set B	[3, 5], 5 + 5	0.0025 ms	$O(n + m)$

21

This demonstrates Dynamic set operations. Operations such as `add`, `remove`, and `contains` are displayed; these operations either change a set or test whether a value is present.

From a Big-O perspective, the set behavior differs from sorting or indexing behavior, with the point being not to find where a value sits in a sequence, but to determine whether the value belongs to the collection.

Build Set Lab: Static Operations

Demonstrate static set operations that derive new results from Set A and Set B.

The screenshot shows three panels for static set operations. Each panel includes a title, input sets, a resulting set, and a time taken to compute it.

Set A difference Set B

Set A	Set B	Result	Time	Complexity
[1, 2, 8]	[5, 9]	[1, 2, 8]	0.0013 ms	$O(n + m)$

Set B difference Set A

Set A	Set B	Result	Time	Complexity
[1, 2, 8]	[5, 9]	[5, 9]	0.0010 ms	$O(n + m)$

Set A symmetric difference Set B

Set A	Set B	Result	Time	Complexity
[1, 2, 8, 4, 9]	[5, 9]	[1, 2, 8, 4, 5]	0.0021 ms	$O(n + m)$

Key Takeaway

- Static operations compare two sets at once.
- Union, intersection, difference, and symmetric difference answer relationship questions.
- The screen shows membership logic rather than sequence order.

22

This demonstrates static set operations. They compare two collections and produce a result from them. The static operations are union, intersection, difference, and symmetric. These operations do not depend on position, but they depend on relationships between the two source sets.

Bubble Sort Lab: Trace Setup

Introduce the Bubble Sort lab and its traceable workflow for sorting integer datasets.

Algorithm and Data Structure Comparison Tool
CS 426: Design and Analysis of Algorithms (Professor Dr. Jonathan Gordon) (Spring 2020) (Student) (Developer) (Records)

Overview Search Comparisons Linked Structures Tree Map Hash Priority Single Algorithms **Bubble Quickselect Sets** Comprehensive Analysis

Bubble Quickselect Sets

Overview Build Sort Lab **Bubble Sort Lab** Quickselect Lab Benchmark Lab Workflow Analysis

Bubble Sort Lab

Dataset Source: Manual Dataset:
 Dataset Size: Manual Dataset:
 Random Seed: Manual Dataset:
 Run Bubble Sort

Bubble Sort complete

Initial Size	Comparisons	Swaps	Writes	Elapsed
10	44	22	44	0.0218 ms

Property	Value
Stable	Yes
Comparisons in place	Yes
Index based	[0]
Early exit used	Yes

Original Dataset **Sorted Dataset**

Key Takeaway

- Bubble Sort traces adjacent comparisons and swaps.
- The setup makes the input and execution path visible.
- This lab favors clarity over production efficiency.

23

The Bubble Sort Lab feature demonstrates bubble sort operations within a lab. This slide shows the set-up portion of the feature. The algorithm continually compares neighboring values and swaps them when they are in the wrong order. Bubble Sort is valuable for educational purposes and comparison with other sorting algorithms rather than for real-world applications.

Quickselect Lab: Valid Rank Selection

Demonstrate Quickselect as a rank-selection algorithm rather than a full sorting algorithm.

Algorithm and Data Structure Comparison Tool
©2020 Design and Analysis of Algorithms | Professor Dr. Jochen Hees | Spring 2020 | Student: Alexander Hees

Overview Search Comparison Linked Lists Trees Hash Priority Graph Algorithms **Quickselect Sort** Complexity Analysis

Bubble Quickselect Sets

Overview Bubble Sort Lab **Quickselect Lab** Merge Sort Lab Quickselect Lab Binary Search Lab

Quickselect Lab

Dataset Source: Manual Dataset:
 Dataset Size:
 Random Seed:
 Kth Smallest Value:
Run Quickselect

Quickselect complete.

Input Size	k	Divided Rank	Selected Value	Correct
10	1	0	275	Yes

Array:
 Expected value from sortedDataset: 12
 Comparisons: 10
 Swaps: 4
 Runtime: 8
 Execution Time: 85528 ns

Original Dataset **Partitioned Dataset**

Key Takeaway

- Quickselect returns the kth-smallest value.
- It avoids sorting the entire dataset when only one rank matters.
- The screen demonstrates selection as a different problem from sorting.

25

The Quickselect Lab feature demonstrates bubble sort operations within a lab.

This slide shows the selection validation of the feature Quickselect, which is used to answer a different question than a full sorting algorithm, like Bubble sort; its main purpose is to find a requested rank, such as the kth-smallest value, without having to fully to have to order the dataset.

Quickselect has an average time complexity of $O(n)$ as it recursively partitions the dataset and only selects and compares the side of the partition that can still contain the requested rank, rather than comparing each value in the dataset.

Quickselect is best suited for workloads that require a specific rank rather (or a single element rather) than a fully sorted result.

Quickselect Lab: Validation

Show how the Quickselect lab validates invalid rank input and preserves correct algorithm behavior.

Key Takeaway

- Invalid k values are handled with clear validation.
- The app protects the algorithm from impossible rank requests.
- This screen shows user-facing robustness, not only runtime behavior.

```

Quickselect Partition Trace
Search bounds: 0 to pivot index 9, pivot value 805
Keep 770 at index 0, it is a pivot 805.
Keep 913 at index 1, it is a pivot 805.
Keep 913 at index 2, it is a pivot 805.
Keep 275 at index 3, it is a pivot 805.
Keep 283 at index 4, it is a pivot 805.
Keep 587 at index 5, it is a pivot 805.
Keep 425 at index 6, it is a pivot 805.
Leave 806 on the right side, it is > pivot 805.
Move 275 < 805: swap indices 8 and 9.
Place pivot 805: swap indices 8 and 9.
Print final position 8: current data: [770, 803, 503, 275, 393, 587, 425, 277, 805, 906]
Target rank: 0 is left of pivot, search 0-7 next.
Search bounds: 0 to pivot index 7, pivot value 277.
Leave 770 on the right side, it is > pivot 277.
Leave 803 on the right side, it is > pivot 277.
Leave 503 on the right side, it is > pivot 277.
Move 275 < 277: swap indices 3 and 5.
Leave 389 on the right side, it is > pivot 277.
Leave 587 on the right side, it is > pivot 277.
Leave 425 on the right side, it is > pivot 277.
Place pivot 277: swap indices 1 and 7.
Print final position 1: current data: [275, 277, 503, 770, 393, 587, 425, 803, 805, 906]
Target rank: 0 is left of pivot, search 0-6 next.
Search bounds: 0 to pivot index 0, pivot value 275.
Print 275 already belongs at index 0.
Print final position 0: current data: [275, 277, 503, 770, 393, 587, 425, 803, 805, 906]
Target rank: 0 found with value 275.
  
```

28

This slide shows the value validation process of the lab feature.

It verifies that A requested rank falls within the bounds of the dataset being processed. When it does not, the app prevents the operation and provides a message.

Note that the app is not only implementing the algorithm itself. It is also handling the workflow by managing edge cases so that results are valid and understandable.

Bubble Quickselect Sets Written Analysis: Sorting and Selection

Explain how the written analysis connects Bubble Sort and Quickselect to different kinds of ordering problems.

Algorithm and Data Structure Comparison Tool

2022-2023 Algorithm and Data Structure Comparison Tool. Copyright © 2022. All Rights Reserved.

Overview: Search Operations, Linked Structures, Tree Traversal, Graph Algorithms, **Sorting and Selection**, Complexity Analysis.

Bubble Quickselect Sets

Overview: Bubble Sort, Quickselect, Set Operations, Complexity Analysis.

Written Analysis

Bubble Sort, Quickselect, and Set Operations Analysis

Overview

Sorting, Selection, and Set Operations are important concepts in computer science that often arise in different kinds of problems. In this project, we will explore the connections between these concepts and how they relate to different kinds of ordering problems. We will also explore the connections between these concepts and how they relate to different kinds of ordering problems.

The purpose of this project is to explore how these concepts relate to different kinds of ordering problems. We will explore the connections between these concepts and how they relate to different kinds of ordering problems. We will also explore the connections between these concepts and how they relate to different kinds of ordering problems.

From a Big O perspective, the Set ADT has a time complexity of $O(n^2)$ for the `insert` and `remove` operations, and $O(n)$ for the `find` operation. The Bubble Sort algorithm has a time complexity of $O(n^2)$ for the `sort` operation. The Quickselect algorithm has a time complexity of $O(n)$ for the `select` operation.

In practice, the Set ADT is often implemented using a hash table or a binary tree. The Bubble Sort algorithm is often implemented using a simple loop. The Quickselect algorithm is often implemented using a recursive function.

Set ADT Behavior

A Set ADT is an abstract data type that represents a collection of unique elements. It supports the following operations:

- `insert(x)`: Add element `x` to the set.
- `remove(x)`: Remove element `x` from the set.
- `find(x)`: Check if element `x` is in the set.
- `size()`: Return the number of elements in the set.

From a Big O perspective, the Set ADT has a time complexity of $O(n^2)$ for the `insert` and `remove` operations, and $O(n)$ for the `find` operation. The Bubble Sort algorithm has a time complexity of $O(n^2)$ for the `sort` operation. The Quickselect algorithm has a time complexity of $O(n)$ for the `select` operation.

In practice, the Set ADT is often implemented using a hash table or a binary tree. The Bubble Sort algorithm is often implemented using a simple loop. The Quickselect algorithm is often implemented using a recursive function.

Table 1: Comparison of Sorting and Selection Algorithms

Algorithm	Time Complexity	Space Complexity	Stability
Bubble Sort	$O(n^2)$	$O(1)$	Yes
Quickselect	$O(n)$	$O(1)$	No
Set Operations	$O(n^2)$	$O(n)$	Yes

Note: The implementation of the Set ADT is not shown in this table. The implementation of the Set ADT is not shown in this table.

Key Takeaway

- Bubble Sort is readable but inefficient for large input.
- Quickselect is better when only one rank is needed.
- The analysis separates full ordering from targeted selection.

28

The Bubble Quickselect Sets Written Analysis Lab feature displays a written analysis of Bubble Sort, Quickselect, and the set data structure.

This slide shows the analysis of the Bubble Sort and Quickselect algorithms of the feature

- Bubble Sort is used to produce a fully ordered list.
- Quickselect is used to find one requested rank.

Those are related problems, but with different purposes.

In practical terms, this slide reinforces a central course idea. The best algorithm depends on the question being asked. If full ordering is required, a sorting algorithm is necessary. If only one rank is needed, Quickselect may be the more direct solution.

Bubble Quickselect Sets Written Analysis: Set Operations

Explain how the written analysis places set operations beside search, sorting, and selection as a different type of data question.

Key Takeaway

- Set operations focus on membership and uniqueness.
- Hashing can make membership checks efficient on average.
- The analysis explains why sets differ from sequence structures.

Bubble Sort

Bubble Sort repeatedly compares adjacent values and swaps them when the left value is greater than the right value. The Bubble Sort reference explains that this kind of algorithm is not recommended because it only works with one neighboring pair at a time. In this project, the implementation also uses a nearly self-optimization. If one pass completes with no swaps, the list is already sorted and the algorithm does not reevaluate.

From [aditya@perspective](#), Bubble Sort has a mixed profile. The best case is [O\(n\)](#) when the data is already sorted and early exits make the algorithm after one pass. The average and worst cases remain [O\(n²\)](#) because the nested loop design can require repeated scans throughout most of the list. Auxiliary space is [O\(1\)](#), which makes Bubble Sort conceptually its great

input data, the search rank moves strongly up that description. The average number for *average base* is 4.00, goes from 3.00000 to 3.00000 at 30 items to 3.00000 at 40 items and 2.00000 at 50 items. That is, a 50% increase from the smaller to the largest tested size, the input data also multiplies a great deal, at 50 items, sorted input finds in 3.00000 sec., while reverse sorted input requires 3.00000 sec. (which is about 30% slower). These results make the early exit effect just as strong.

is not already close to sorted order.

Quickselect Analysis

Quickselect finds a pivot, sorts without fully sorting the dataset, it partitions the active region around a pivot, then continues only into the side that can still contain the requested rank. This project, in its original version, uses a deterministic (Lomuto-style) partition with the rightmost active value as the pivot. That choice is useful for classroom use reproducibility, but it can create predictable weak cases.

in practice, the benchmark results show both the strengths and the limitation of this implementation. The average runtime for `getNearestNeighbours` grows from 0.0000046 s at 20 neighbours to 0.0000046 s at 1,000,000 neighbours and 0.0000046 s at 1,000,000 neighbours. This is because the algorithm uses a nearest neighbour search, which is not efficient for large numbers of neighbours.

The dataset reader was also very important; it took items from disk sequentially instead of randomly as we had done before.

[illegible]

Another practical point is also important. In this Python environment, Quakeblast is not faster than the `numpy.linalg.solve()` baseline. In fact, the average Quakeblast performance is slower as well. This suggests that the current implementation of Quakeblast is not as efficient as the `numpy.linalg.solve()` baseline. What does this mean for the overall idea of using Quakeblast?

[illegible]

Comparative Big-O Analysis

Fig 8 Mutation is useful in this project because it explains why the relative rankings in the benchmark data are not accidental. Bubble Sort does repeated adjacent comparisons and therefore sorts quadratically on average. Quicksort tries to avoid falling into and therefore has linear average behavior, but it still suffers when poor choices are made. Insertion Sort does better on small data, while Shell Sort does best overall. These are very different algorithms. These are very different algorithms. These are very different algorithms.

Table 2
Theoretical Complexity Comparison of the Core Module # Operations

Structure or Model/Tool	Goal	Average	Worst	Space	Key implementation note
-------------------------	------	---------	-------	-------	-------------------------

Algorithm	Input	Output	Complexity	Remarks
Self-organizing Map (SOM) / Kohonen	Set of input vectors	Neural network with competitive learning	$O(N \times V)$	Multi-layered feedforward with competitive learning
Interpretation / Information / Differences	Not usually expressed separately	$O(N \times V)$ with neural training, more for learning weights	$O(N \times V)$	Returns a new derived set and shows weights and bias
Build for test with only	$O(N)$	$O(N \times V)$	$O(N)$	Adjust comparison learning with given build and test
Discretized	$O(N)$	$O(N)$	$O(N)$	High-level with symbolic notation for built model solution
Pattern search / Feature selection	Order on highly ordered inputs, but generally treated as $O(N \times \log V)$	$O(N \times \log V)$	$O(N \times \log V)$	Dependent on representation details
				Relates to Feature Selection and/or Classification and Validation

Note: The table reflects the specific implementations used in this project, not every possible implementation of each, sorting, or selection.

Table 2 matches the observed growth pattern's worst average `worstInsertSort`, `worstMergeSort`, `worstQuickSort`, `worstRadixSort`, `worstSelectionSort`, `worstShellSort`, `worstBubbleSort`, `worstHeapSort`, `worstCountingSort`, `worstBucketSort`, `worstRadixSort`, `worstTimSort`, while average `averageInsertSort`, `averageMergeSort`, `averageQuickSort`, `averageRadixSort`, `averageSelectionSort`, `averageShellSort`, `averageBubbleSort`, `averageHeapSort`, `averageCountingSort`, `averageBucketSort`, `averageRadixSort`, `averageTimSort`.

10.16. Inverted, cosine curve fits best growth of 10.16c, and cosine curve fit sublinear growth 10.16d. These different possibilities are exactly what should be expected when one workload is quadratic, another is nearly linear but input sensitive, and the built-in baseline comes from a heavily optimized library implementation.

This slide shows the set operations portion of the analysis. It explains that sets evaluate uniqueness, membership, and collection relationships. These operations are used to know whether a value belongs to a collection or how two collections relate to each other.

In practical terms, sets help answer questions that search, sorting, and rank selection do not answer as directly.

Bubble Quickselect Sets Written Analysis: Learning Outcomes

Summarize how the Module 8-native features extend the integrated portfolio and deepen the final learning outcomes.

Benchmark Results

The benchmark table measured runtime for every major workload at sizes 10, 100, and 1000 across five dataset scenarios: random, sorted, reverse sorted, slightly sorted, and partially sorted.

Workload Groups

- Bubble Sort (10, 100, 1000)** measures full dataset comparison sorting with early exit.
- Python sorted (10, 100, 1000)** measures the built-in fastest sorting for comparison sorting.
- Quickselect (10, 100, 1000)** measures partition-based kth smallest selection.
- Python sorted (10, 100, 1000)** measures the assignment baseline of selecting the kth smallest value by fully sorting first.

Table 1
Average Runtime by Operation and Size

Size	Bubble Sort full sort	Python sorted full sort	Quickselect 10th selection	Python sorted 10th selection
10	0.000796 sec	0.000847 sec	0.000792 sec	0.000803 sec
100	0.002222 sec	0.002243 sec	0.002028 sec	0.002223 sec
1000	0.002680 sec	0.002693 sec	0.002688 sec	0.002693 sec

Table 2
Dataset Scenario of Size 100

Dataset type	Bubble Sort full sort	Quickselect 10th selection
random	0.002694 sec	0.002670 sec
sorted	0.000847 sec	0.000803 sec
reverse sorted	0.002694 sec	0.002670 sec
slightly sorted	0.002694 sec	0.002670 sec
partially sorted	0.002694 sec	0.002670 sec

The benchmark results show that the built-in Python functions are the strongest practical performers in this project. At 100 items, Python sorted (10, 100) averages only 0.002222 sec, while Bubble Sort (10, 100) averages 0.002222 sec. This means Bubble Sort is about 100 times slower than the built-in full sort baseline at the largest tested size. The same pattern appears in 10th selection. At 100 items, Quickselect (10, 100) averages 0.002028 sec, while Python sorted (10, 100) averages 0.002223 sec. However, the comparison makes the entire algorithm's full runtime. Quickselect is far more favorable than Bubble Sort when the task is selection rather than full sorting. On random input of size 100, Bubble Sort requires 0.002694 sec, while Quickselect needs only 0.002670 sec. On already sorted input of the same size, Bubble Sort requires 0.000847 sec, while Quickselect needs only 0.000803 sec. These results support the main algorithm lesson of the module: finding one rank is a different problem than sorting everything, and the choice of algorithm should reflect that difference.

The benchmark results also make merge sortivity easy to see. Bubble Sort is helped enormously by already sorted data because it only needs to swap pairs. Quickselect, on the other hand, is hurt by sorted and reverse sorted data in this implementation because the rightmost pivot repeatedly creates poor partitions. These contrasting results are especially useful for teaching because they show that best case, average case, and worst case behavior are not abstract ideas. They appear directly in measured runtime, comparison counts, swap counts, and write counts.

Conclusion

The results of this project show that the chosen implementations behave for any algorithmic problem. The built-in Python functions are the strongest practical performers in this project. At 100 items, Python sorted (10, 100) averages only 0.002222 sec, while Bubble Sort (10, 100) averages 0.002222 sec. This means Bubble Sort is about 100 times slower than the built-in full sort baseline at the largest tested size. The same pattern appears in 10th selection. At 100 items, Quickselect (10, 100) averages 0.002028 sec, while Python sorted (10, 100) averages 0.002223 sec. However, the comparison makes the entire algorithm's full runtime. Quickselect is far more favorable than Bubble Sort when the task is selection rather than full sorting. On random input of size 100, Bubble Sort requires 0.002694 sec, while Quickselect needs only 0.002670 sec. On already sorted input of the same size, Bubble Sort requires 0.000847 sec, while Quickselect needs only 0.000803 sec. These results support the main algorithm lesson of the module: finding one rank is a different problem than sorting everything, and the choice of algorithm should reflect that difference.

The benchmark results also make merge sortivity easy to see. Bubble Sort is helped enormously by already sorted data because it only needs to swap pairs. Quickselect, on the other hand, is hurt by sorted and reverse sorted data in this implementation because the rightmost pivot repeatedly creates poor partitions. These contrasting results are especially useful for teaching because they show that best case, average case, and worst case behavior are not abstract ideas. They appear directly in measured runtime, comparison counts, swap counts, and write counts.

What this screen shows

- The module reinforces theory through implementation and measurement.

- Traces, benchmarks, and validation make behavior inspectable.

- Learning outcomes connect algorithm choice to problem type.

30

This slide shows the main learning outcomes that analysis provides.

It explains how:

- `CourseSet` adds set algebra and membership logic.
- Bubble Sort adds a traceable sorting demonstration.
- Quickselect adds efficient rank selection.

Comprehensive Analysis: Executive Overview

Introduce the comprehensive analysis as the integrated summary of the full portfolio's theory, benchmarks, and recommendations.

Algorithm and Data Structure Comparison Tool

©2023 Deep and Advanced Algorithm Portfolio. All rights reserved. Contact: support@deepalgorithms.com

Home | Performance | Comprehensive Analysis | Search Algorithms | Sorting Algorithms | Data Structures | **Comprehensive Analysis**

Comprehensive Analysis

Comprehensive Performance Analysis

Executive Overview

The purpose of this comprehensive analysis is to provide a detailed overview of the performance of various algorithms and data structures. This report includes a comparison of different algorithms, a summary of their strengths and weaknesses, and a set of recommendations for their use in different contexts. The analysis is based on a series of experiments and benchmarks that were conducted over a period of several weeks. The results of these experiments are presented in the following sections.

Search Algorithms: Linear Search vs Binary Search

Search algorithms are a fundamental part of many applications. They are used to find a specific element in a collection of data. There are two main types of search algorithms: linear search and binary search. Linear search is a simple algorithm that checks each element in the collection until it finds the element it is looking for. Binary search is a more complex algorithm that uses a divide-and-conquer strategy to find the element. In this section, we will compare the performance of these two algorithms.

Sorting and Selection: Bubble Sort, Sorting Context, Quickselect

Sorting algorithms are used to arrange a collection of data in a specific order. There are many different sorting algorithms, but two of the most common are bubble sort and quickselect. Bubble sort is a simple algorithm that compares adjacent elements and swaps them if they are in the wrong order. Quickselect is a more complex algorithm that uses a partitioning strategy to select the k-th smallest element. In this section, we will compare the performance of these two algorithms.

Key Takeaway

- The final analysis integrates results across all modules.
- It brings benchmarks, tradeoffs, and recommendations into one view.
- The screen turns separate labs into a portfolio-level argument.

31

The Comprehensive Analysis feature provides a full written analysis of all the algorithms and data explored by the app.

It brings together the written analyses, benchmark, and recommendation logic from all the app features. The purpose of the analysis is to examine the different approaches both theoretically and practically. In other words, it is a comparison system that explains how different growth rates, data structures, and workload influence design and technical decision-making.

This slide displays the executive overview of the analysis, a comparison of linear search vs binary search algorithms, and a comparison of sorting and selection using the Bubble Sort and Quickselect algorithm.

Comprehensive Analysis: Data Structure Tradeoffs

Compare the major data structure families and explain how their strengths depend on workload and implementation details.

Hash Tables and Priority Queues

Hash tables and priority queues are important because they solve different kinds of retrieval problems. A hash table is used when the main goal is fast key-based lookup. A priority queue is used when the main goal is to repeatedly remove the highest-priority or lowest-priority item. In the integrated portfolio, this section combines the hash priority module's written analysis with the sparse benchmark artifacts from the same module.

It is important to note that the same data structure idea can behave very differently depending on how it is implemented. Hash tables depend heavily on hash quality and collision behavior, while heaps depend on maintaining parent-child ordering in a nearly complete binary tree (CSU Global, n.d.-b; Lynch & Muth, 2018). The saved benchmark results support that point clearly. In particular, they show how strongly collision buildup and full linear scans influence real performance.

From a Big-O perspective, the hash table is the strongest general lookup structure in this portfolio under healthy conditions. Insert, search, and delete are all average-case $O(1)$ because the workload usually only needs one bucket and a short chain. The worst case is $O(N)$ because too many keys can fall into the same bucket. The hash-table reference explains why this kind of associative storage can be very fast when indexing is computed efficiently (Lecture Hand, n.d.-c).

In practice, the saved results strongly support that description. At 11,458 items, hash-table search finishes in roughly 0.2134 s for hits, 0.3534 s for misses, and 0.1025 s for mixed queries. Over the same workload sizes and query modes, the linear-search baseline requires 26.4424 s, 76.2338 s, and 34.5457 s. These results show that the hash table is the strongest lookup structure in the portfolio when the key distribution is healthy.

The collision results are just as important because they show the limit of hashing. Formed-collision search hits at size 100 already rise to 0.1001 s, average per query with 161 collisions recorded, and the CTA-5 analysis shows that this becomes much worse at larger sizes. Separate chaining preserves correctness, but it does not remove the cost of very poor bucket distribution. In practical terms, the hash table is extremely strong, but only when the hash function and collision distribution remain favorable (CSU Global, n.d.-b; Lynch & Muth, 2018).

Priority queues show a different pattern. From a Big-O perspective, push remains $O(1)$ while insert and extract_top are $O(\log V)$ because the heap only needs to restore order along one path. Search-by-label and delete-by-label are $O(1)$ because the heap property does not provide direct access to an arbitrary item. In practice, the saved results show that clarity. At 15,000 items, search_by_label stays near 0.1 to 0.4 s, push remains effectively constant at 0.0016 s, and extract_top stays at about 76.0 s. These results show that the priority queue is very effective when the workload needs most access, but much less effective when the workload needs general lookup.

Graphs: Representations, BFS, DFS, Dijkstra

Graphs are important because they model relationships between objects. In this portfolio, the graph module compares two graph representations, adjacency_list and adjacency_matrix , and also evaluates BFS, DFS, Dijkstra, and Bellman-Ford. The purpose of this section is to show how graph density, representation choice, and edge rules affect both performance and correctness.

It is important to note that representation choice depends strongly on the workload. The list-based model stores real neighbor relationships, which makes it a strong fit for sparse graphs. The matrix-based model allocates a row-column position for each source-target pair, which can help in dense, stable workloads where direct edge checks are common (CSU Global, n.d.-d; Lynch & Muth, 2018). The saved benchmark results support that point clearly.

From a Big-O perspective, adjacency-list storage uses $O(V + E)$ space, while adjacency-matrix storage uses $O(V^2)$ space. BFS and DFS fit naturally with the list-based model because traversal requests the stored neighbors of the current vertex instead of scanning many empty relationships. Matrix storage has a clear theoretical advantage for direct adjacency checks because a known source-target pair can be checked in $O(1)$ time (CSU Global, n.d.-d; Lynch & Muth, 2018).

In practice, the saved graph results strongly favor adjacency lists on sparse workloads. The list-based representation wins 33 of 34 sparse operation size buckets. At 100 sparse vertices, the saved results show list-based storage winning built, neighbor scan, BFS, DFS, Dijkstra, Bellman-Ford, and edge removal. The matrix remains important, but it is more specialized. The densest and heaviest loads show that it can become competitive or superior for some specific path cases, especially dense 20-vertex and dense Bellman-Ford at the largest benchmark size.

The algorithm layer adds an important correctness lesson. BFS and DFS are both traversal tools, but they visit vertices in different orders. Dijkstra and Bellman-Ford are both shortest-path tools, but they are not interchangeable. The saved CTA-7 analysis uses Dijkstra for the positive-weight Dens-to-Hall route and Bellman-Ford for the negative-cost demo because correctness depends on the edge rules, not just on speed (Liu & Mangen, 2018). In practical terms, the graph section shows that the best-performing representation or algorithm only matters after the chosen method is valid for the graph's density and edge semantics.

Key Takeaway

- Structures optimize access patterns, not generic speed.
- Stacks, queues, maps, hashes, heaps, and graphs serve different workloads.
- The table supports comparison across the whole course.

33

This slide displays an analysis of:

- Hash tables and priority queues data structures
- Graphs data structure and the BFS, DFS, and Dijkstra algorithms

The analysis explains that

- Linked structures manage endpoint work, shifting, and traversal tradeoffs.
- Trees and maps balance ordered lookup and height sensitivity.
- Hash tables handle direct key-based access and collision effects.
- Priority queues are used for priority removal.
- Graphs are a relationship storage and traversal strategy.

The analysis explains that data structures are selected by workload requirement, not by a simple ranking from best to worst.

Comprehensive Analysis: Recommendation Matrix

Present the recommendation matrix that converts the full portfolio evidence into actionable algorithm and data structure choices.

Key Takeaway

- The recommendation matrix converts evidence into decisions.
- Each row maps a workload to a preferred structure or algorithm.
- The matrix makes tradeoffs usable beyond the demo app.

Cross-Module Benchmark and Big-O Comparison

Big-O notation is useful in this integrated portfolio because it explains why the benchmark rankings are not accidental. Lower growth algorithms and better aligned structures usually scale better, but the used and effects also show that tree nodes, tree shapes, set-like behavior, and graph density can change the practical result. The table below summarizes the dominant operation modes for each major area of the portfolio.

Table 1 Cross-Module Benchmark and Big-O Comparison Summary				
Domain	Structure or Algorithm	Typical Strength	Typical Cost Model	Main Limitation
Search	Linear search	Works on unsorted data consistently	$O(N)$	Scalable directly with dataset size
Search	Binary search	Dependent search on sorted data	$O(\log N)$	Requires sorted order
Sorting	Bubble Sort	Trivial full sorting with early exit	Best: $O(N^2)$, Average: $O(N^3)$	Very sensitive to unsorted input
Sorting	Merge Sort	Large N-sort workloads	$O(N \log N)$	More complex than reference sorts
Selection	Quickselect	Find one rank without full sort	Average: $O(N)$, Worst: $O(N^2)$	Poor worst-case
Set	Set-based membership	Uniqueness and membership	Average: $O(1)$	Dependent on hashing quality
Set	Union/Intersection/Difference	Derives relationships between collections	$O(N + M)$	Must scan both operands
Linked Structures	Block	Random item access	Constant, friendly, workload dependent $O(1)$ time	Model can regress to $O(N)$ in this implementation
Linked Structures	Queue	FIFO ordered traversal	From shifting full cost	30% 100% vs. enqueue at 10,000
Linked Structures	Deque	First and access	End dependent, mixed $O(1)$ and $O(N)$	Some $O(N)$ very different scenarios
Linked Structures	Linked List	End-point mutation without a flag	Traversal heavy operations $O(N)$	Hidden access requires traversal
Tree/Maps	BST/TreeMap	Ordered search and traversal	Typical $O(\log N)$, if height stays short	Can degenerate to $O(N)$ when skewed
Tree/Maps	HashMap	Simple linear baseline	$O(1)$	No structural overhead
Hashing	Hash table	Direct key lookup	Average $O(1)$	Collisions can push toward $O(N)$
Priority	Priority heap priority queue	Dependent cost access	Java: $O(N)$, Java/linked: $O(N \log N)$	Arbitrary label search $O(N)$
Graphs	Adjacency list	Sparse graph storage and traversal	Space $O(N + E)$	Less suitable for dense graph checks
Graphs	Adjacency matrix	Dense edge checks and constant time workloads	Space $O(N^2)$	Regular all possible paths

Note: This table summarizes the dominant strength, cost model, and practical limitation for each major structure or algorithm area. Table 2 shows how important patterns. First, no single complexity class dominates every area of the portfolio because the modules solve different kinds of problems. Second, the strongest practical performers are the structures that align closely with the main workload. Binary search is strongest for repeated search on sorted data. Hash tables are strongest for direct key lookup. Maps are strongest for repeated key priority access. Adjacency lists are strongest for sparse traversal heavy graphs. These patterns match the practical recommendations made throughout the earlier modules.

This slide displays a Recommendation Matrix. The matrix provides a summarized analysis of the algorithms and data structures that can be easily translated into decisions. It covers repeated search, sorted lookup, full ordering, kth-rank selection, uniqueness, FIFO access, LIFO access, priority removal, sparse traversal, or dense adjacency checks. The matrix does not treat one algorithm as the winner in every category. Instead, it shows that different tools are useful because they optimize different operations under different constraints.

Comprehensive Analysis: Technical Achievements and Course Learning

Close the presentation by summarizing the technical achievements, integration work, and final CSC506 learning outcomes.

Key Takeaway

- The final summary of the complete technical achievement.

- The project combines implementation, testing, visualization, and analysis.

- The portfolio demonstrates theory, measurement, and recommendation together.

Practical Recommendation Matrix

This matrix summarizes the technical achievements and course learning outcomes. The matrix is organized into four columns: Recommendation, Recommendation, Recommendation, and Recommendation.

Recommendation	Recommendation	Recommendation	Recommendation
Recommendation 1	Recommendation 2	Recommendation 3	Recommendation 4
Recommendation 5	Recommendation 6	Recommendation 7	Recommendation 8
Recommendation 9	Recommendation 10	Recommendation 11	Recommendation 12
Recommendation 13	Recommendation 14	Recommendation 15	Recommendation 16
Recommendation 17	Recommendation 18	Recommendation 19	Recommendation 20
Recommendation 21	Recommendation 22	Recommendation 23	Recommendation 24
Recommendation 25	Recommendation 26	Recommendation 27	Recommendation 28
Recommendation 29	Recommendation 30	Recommendation 31	Recommendation 32
Recommendation 33	Recommendation 34	Recommendation 35	Recommendation 36
Recommendation 37	Recommendation 38	Recommendation 39	Recommendation 40
Recommendation 41	Recommendation 42	Recommendation 43	Recommendation 44
Recommendation 45	Recommendation 46	Recommendation 47	Recommendation 48
Recommendation 49	Recommendation 50	Recommendation 51	Recommendation 52
Recommendation 53	Recommendation 54	Recommendation 55	Recommendation 56
Recommendation 57	Recommendation 58	Recommendation 59	Recommendation 60
Recommendation 61	Recommendation 62	Recommendation 63	Recommendation 64
Recommendation 65	Recommendation 66	Recommendation 67	Recommendation 68
Recommendation 69	Recommendation 70	Recommendation 71	Recommendation 72
Recommendation 73	Recommendation 74	Recommendation 75	Recommendation 76
Recommendation 77	Recommendation 78	Recommendation 79	Recommendation 80
Recommendation 81	Recommendation 82	Recommendation 83	Recommendation 84
Recommendation 85	Recommendation 86	Recommendation 87	Recommendation 88
Recommendation 89	Recommendation 90	Recommendation 91	Recommendation 92
Recommendation 93	Recommendation 94	Recommendation 95	Recommendation 96
Recommendation 97	Recommendation 98	Recommendation 99	Recommendation 100

Conclusion and References

This matrix summarizes the technical achievements and course learning outcomes. The matrix is organized into four columns: Recommendation, Recommendation, Recommendation, and Recommendation.

References

1. [Reference 1]
2. [Reference 2]
3. [Reference 3]
4. [Reference 4]
5. [Reference 5]
6. [Reference 6]
7. [Reference 7]
8. [Reference 8]
9. [Reference 9]
10. [Reference 10]
11. [Reference 11]
12. [Reference 12]
13. [Reference 13]
14. [Reference 14]
15. [Reference 15]
16. [Reference 16]
17. [Reference 17]
18. [Reference 18]
19. [Reference 19]
20. [Reference 20]
21. [Reference 21]
22. [Reference 22]
23. [Reference 23]
24. [Reference 24]
25. [Reference 25]
26. [Reference 26]
27. [Reference 27]
28. [Reference 28]
29. [Reference 29]
30. [Reference 30]
31. [Reference 31]
32. [Reference 32]
33. [Reference 33]
34. [Reference 34]
35. [Reference 35]
36. [Reference 36]
37. [Reference 37]
38. [Reference 38]
39. [Reference 39]
40. [Reference 40]
41. [Reference 41]
42. [Reference 42]
43. [Reference 43]
44. [Reference 44]
45. [Reference 45]
46. [Reference 46]
47. [Reference 47]
48. [Reference 48]
49. [Reference 49]
50. [Reference 50]
51. [Reference 51]
52. [Reference 52]
53. [Reference 53]
54. [Reference 54]
55. [Reference 55]
56. [Reference 56]
57. [Reference 57]
58. [Reference 58]
59. [Reference 59]
60. [Reference 60]
61. [Reference 61]
62. [Reference 62]
63. [Reference 63]
64. [Reference 64]
65. [Reference 65]
66. [Reference 66]
67. [Reference 67]
68. [Reference 68]
69. [Reference 69]
70. [Reference 70]
71. [Reference 71]
72. [Reference 72]
73. [Reference 73]
74. [Reference 74]
75. [Reference 75]
76. [Reference 76]
77. [Reference 77]
78. [Reference 78]
79. [Reference 79]
80. [Reference 80]
81. [Reference 81]
82. [Reference 82]
83. [Reference 83]
84. [Reference 84]
85. [Reference 85]
86. [Reference 86]
87. [Reference 87]
88. [Reference 88]
89. [Reference 89]
90. [Reference 90]
91. [Reference 91]
92. [Reference 92]
93. [Reference 93]
94. [Reference 94]
95. [Reference 95]
96. [Reference 96]
97. [Reference 97]
98. [Reference 98]
99. [Reference 99]
100. [Reference 100]

35

This final slide displays the analysis conclusion and a Recommendation Matrix summarizing all the labs and analyses into a comprehensive recommendation guide.

The project integrates earlier assignments from CSC506.

It includes dataset builders, guided labs, structure visualizations, algorithm traces, benchmark workflows, written analyses, and recommendation guides.

It provides the central learning outcome stating that algorithm analysis is most useful when theory, implementation, measurement, and recommendation are considered together, that is:

- Big-O notation explains growth.
- Benchmarks labs provide empirically measured behavior confirming the theoretical findings from time and space complexity analysis.

Showing that the most important question is not "Which algorithm is best?" is "Which algorithm or data structure is the best fit for this data, this access pattern, and this implementation context?"